Integrando tu Chatbot en una Interfaz Web



Introducción

En este laboratorio, aprenderás a configurar un servidor de backend e integrar tu chatbot en una aplicación web.

Objetivos de aprendizaje

Después de completar este laboratorio, podrás:

- Configurar tu servidor de backend
- Integrar tu chatbot en tu servidor Flask
- Comunicarte con el backend utilizando una página web

Requisitos previos

Esta sección asume que sabes cómo construir el chatbot simple de terminal explicado en el primer laboratorio.

Hay dos cosas que debes construir para crear tu sitio web similar a ChatGPT:

1. Un servidor de backend que aloje tu chatbot
2. Una página web de frontend que se comunique con tu servidor de backend

Sin más preámbulos, ¡comencemos!

The content of this lab is licensed under Apache 2.0

Paso 1: Alojar tu chatbot en un servidor de backend

¿Qué es un servidor de backend?

Un servidor de backend es como el cerebro detrás de un sitio web o aplicación. En este caso, el servidor de backend recibirá solicitudes de tu sitio web, las alimentará a tu chatbot y devolverá la salida del chatbot de nuevo al sitio web, que será leído por el usuario.

Alojar un servidor de backend simple usando Flask

Nota: Considera usar un archivo requirements.txt

`flask` es un marco de trabajo de Python para construir aplicaciones web con Python. Proporciona un conjunto de herramientas y funcionalidades para manejar solicitudes entrantes, procesar datos y generar respuestas, facilitando el funcionamiento de tu sitio web o aplicación.

Requisitos previos

Para todas las interacciones en la terminal en este laboratorio (como ejecutar archivos de python o instalar paquetes), utilizarás la terminal integrada que viene con Cloud IDE. Puedes abrir la terminal presionando:

- `Ctrl + ``, o
- Seleccionando Terminal --> Nueva Terminal en la barra de herramientas en la parte superior de la ventana del IDE a la derecha.

En tu terminal, instalemos los siguientes requisitos:

bash

```
python3.11 -m pip install flask
python3.11 -m pip install flask_cors
```

Run

Configurando el servidor

A continuación, crearás un script que almacene tu código del servidor flask.

Para crear un nuevo archivo de Python, haz clic en `File Explorer`, luego haz clic

The content of this lab is licensed under Apache 2.0

archivo `app.py`.

Veamos cómo implementar un servidor flask simple:

```py
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return 'Hello, World!'

if __name__ == '__main__':
    app.run()
```

Pega el código anterior en el archivo `app.py` que acabas de crear y guárdalo.

En este código:

- Importas la clase `Flask` del módulo `flask`.
- Creas una instancia de la clase `Flask` y la asignas a la variable `app`.
- Defines una ruta para la página de inicio decorando la función `home()` con el decorador `@app.route()`. La función devuelve la cadena '¡Hola, Mundo!'. Esto significa que cuando el usuario visita la URL donde está alojado el sitio web, el servidor backend recibirá la solicitud y devolverá '¡Hola, Mundo!' al usuario.
- La condición `if __name__ == '__main__':` asegura que el servidor solo se ejecute si el script se ejecuta directamente, no cuando se importa como un módulo.
- Finalmente, llamas a `app.run()` para iniciar el servidor.

```bash
python3.11 app.py
```

Run

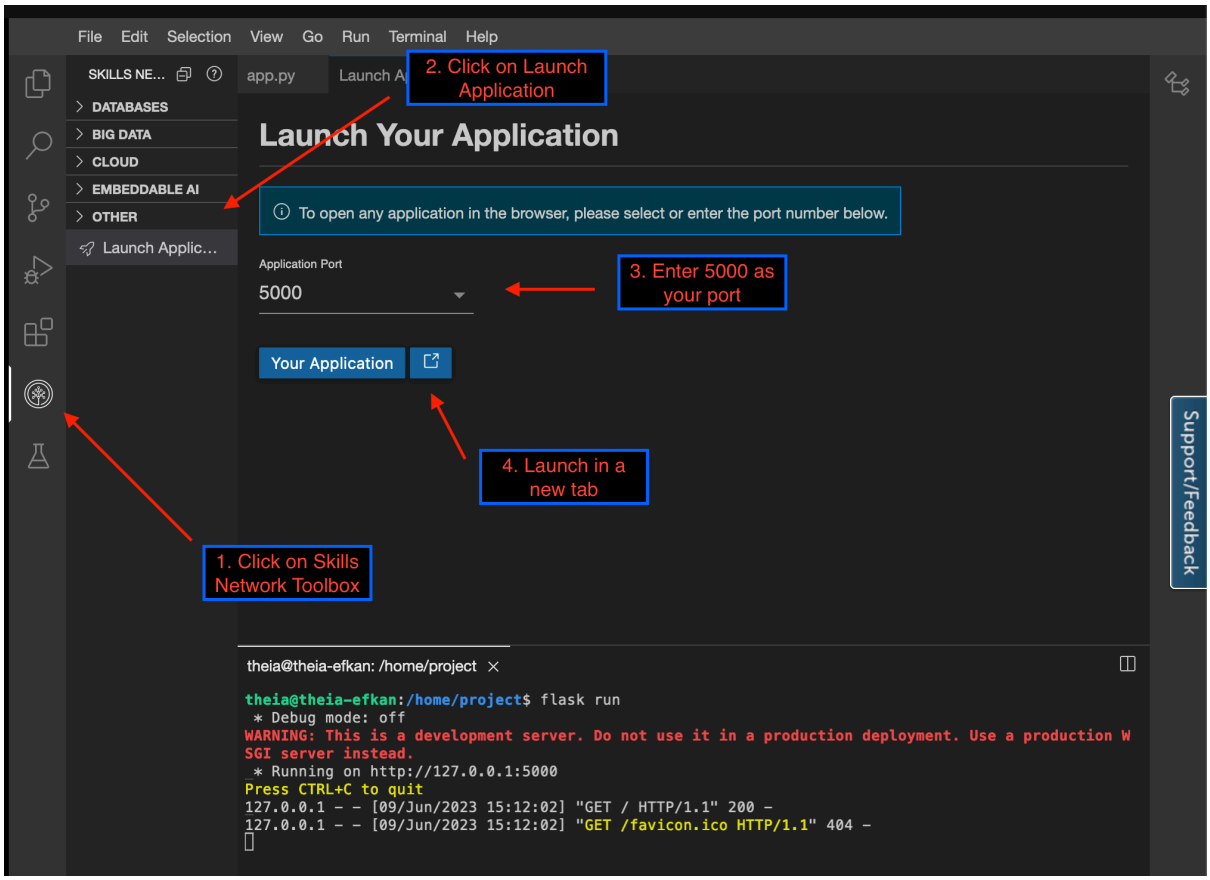
Guarda este código en un archivo de Python, por ejemplo, app.py, y ejecútalo escribiendo `python app.py` en la terminal. Por defecto, Flask aloja el servidor en

The content of this lab is licensed under Apache 2.0

Con este comando, el servidor Flask comenzará a ejecutarse. Si ejecutas este servidor en tu máquina local, puedes acceder a él visitando http://127.0.0.1:5000/ o http://localhost:5000/ en tu navegador web.

Sin embargo, actualmente estás ejecutando este laboratorio en la Nube de Skills Network. Por lo tanto, puedes acceder a tu servidor de la siguiente manera:

1. Navega a la Caja de Herramientas de Skills Network desde la barra de herramientas en el lado izquierdo del IDE
2. Haz clic en "Lanzar Aplicación" en la barra lateral vertical adyacente
3. Ingresa 5000 como tu Puerto de Aplicación
4. Lanza la aplicación en una nueva pestaña del navegador

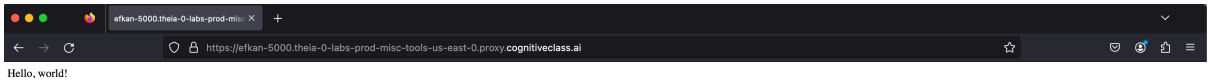


Al realizar los pasos anteriores, has visitado la **URL relativa de localhost** del servidor en la nube.

IMPORTANTE: A lo largo del resto de este laboratorio, te referirás a esta URL como .

The content of this lab is licensed under Apache 2.0

Así es como debería verse:



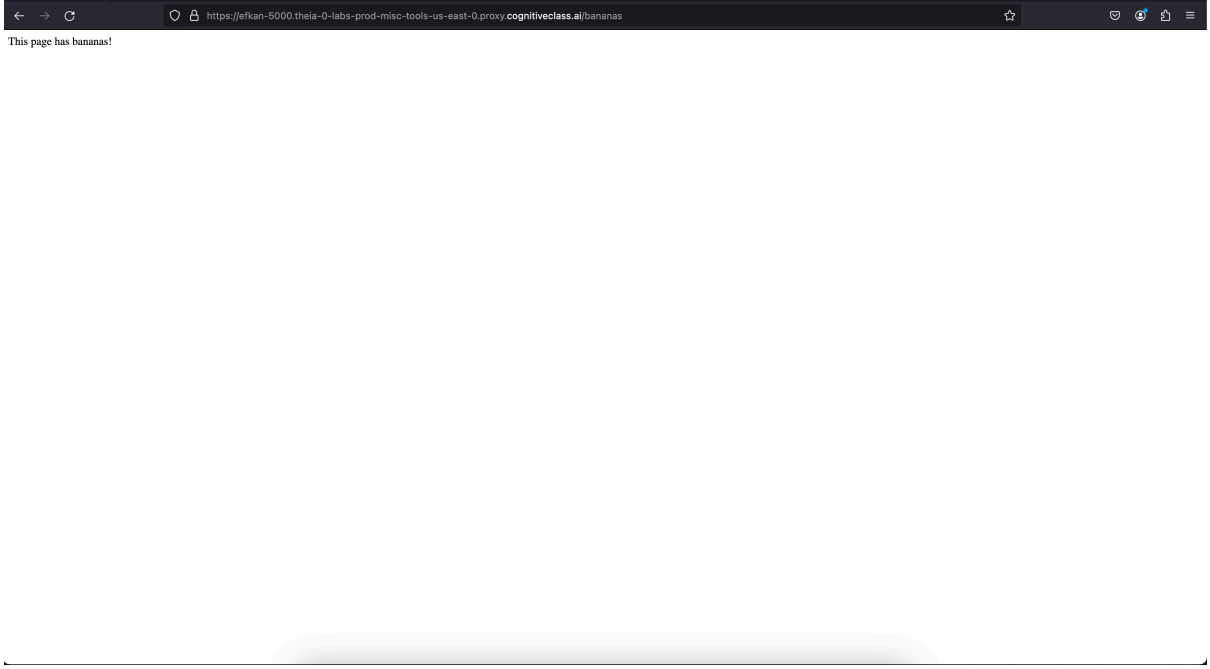
Agreguemos las siguientes rutas para probarlo:

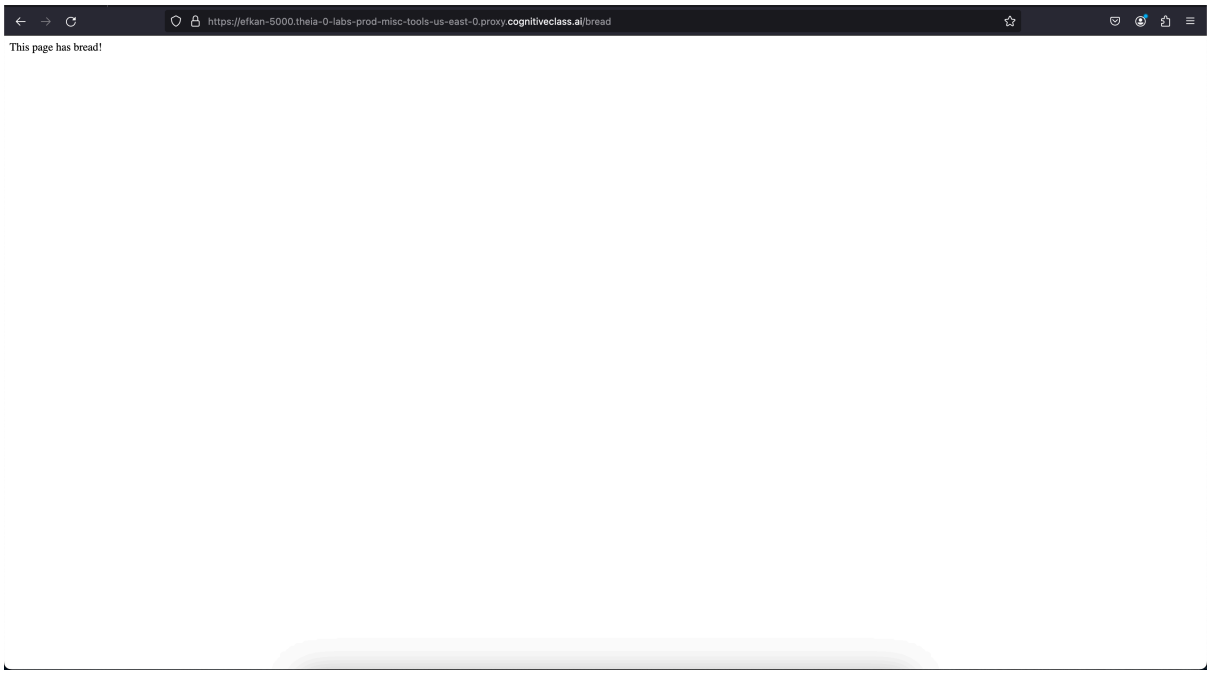
```py
@app.route('/bananas')
def bananas():
    return 'This page has bananas!'

@app.route('/bread')
def bread():
    return 'This page has bread!'
```

Ahora, detengamos nuestra aplicación usando `Ctrl + C` en la terminal y volvamos a ejecutar con `flask run`. Luego, visitemos ambas rutas en /bananas y /bread. Esto es lo que deberías ver:

The content of this lab is licensed under Apache 2.0





Está bien, ahora que has demostrado cómo funcionan las rutas, puedes eliminar estas dos rutas (bananas y bread) de tu `app.py` ya que no las estarás utilizando.

Antes de continuar, también agregarás dos líneas más de código a tu programa para mitigar los errores de CORS, un tipo de error relacionado con hacer solicitudes a dominios diferentes al que aloja esta página web. Modificarás tu código de la siguiente manera:

The content of this lab is licensed under Apache 2.0

```py
from flask import Flask
from flask_cors import CORS     # newly added

app = Flask(__name__)
CORS(app)               # newly added

@app.route('/')
def home():
    return 'Hello, World!'

if __name__ == '__main__':
    app.run()
```

Integrando tu chatbot en tu servidor Flask

Ahora que tienes tu servidor Flask configurado, integremos tu chatbot en tu servidor Flask.

Como se indicó al principio, este laboratorio asume que has completado el primer laboratorio de este proyecto guiado sobre cómo crear tu propio chatbot simple.

Primero, instalarás los requisitos. Fijamos las versiones de las bibliotecas para asegurar que el código se ejecute de manera consistente sin romperse debido a actualizaciones futuras o cambios en las dependencias.

```plaintext
python3.11 -m pip install transformers==4.41.2
python3.11 -m pip install torch==2.2.2
python3.11 -m pip install accelerate==0.30.1
python3.11 -m pip install numpy==1.26.4
```

A continuación, copia el código para inicializar tu chatbot del laboratorio 1 y colócalo en la parte superior de tu script. También debes importar las bibliotecas necesarias para tu chatbot.

```py
from transformers import AutoModelForSeq2SeqLM
from transformers import AutoTokenizer
```

The content of this lab is licensed under Apache 2.0

```py
model_name = "facebook/blenderbot-400M-distill"
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)
conversation_history = []
```

A continuación, necesitarás importar un módulo para leer los datos.

```py
from flask import request
```

Antes de implementar la función real, necesitas determinar la estructura que esperas recibir en la solicitud HTTP entrante.

Definamos tu estructura esperada de la siguiente manera:

```json
{
    "prompt": "message"
}
```

Ahora implementa la función de tu chatbot. Nuevamente, copiarás el código de tu implementación de chatbot del primer laboratorio.

The content of this lab is licensed under Apache 2.0

```py
@app.route('/chatbot', methods=['POST'])
def handle_prompt():
    data = request.get_json()
    input_text = data["prompt"]

    # Keep only recent conversation
    conversation_history[:] = conversation_history[-6:]

    # Create conversation history string
    history = "\n".join(conversation_history)
    prompt = history + f"\nUser: {input_text}\nBot:"

    # Tokenize input
    inputs = tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True,
        max_length=512
    )

    # Generate response
    outputs = model.generate(
        **inputs,
        max_new_tokens=60,
        no_repeat_ngram_size=3,
        repetition_penalty=1.3,
        do_sample=True,
        temperature=0.6,
        top_p=0.85
    )

    # Decode response
    response = tokenizer.decode(outputs[0], skip_special_tokens=True).stri

    # Save conversation
    conversation_history.append(f"User: {input_text}")
    conversation_history.append(f"Bot: {response}")

    return response
```

La única novedad que has hecho hasta ahora es leer el mensaje de la solicitud HTTP. ¡Has copiado todo lo demás de tu implementación anterior del chatbot!

Perfecto, ahora antes de probar tu aplicación, así es como se ve la versión final de

The content of this lab is licensed under Apache 2.0

py

```python
from flask import Flask, request, render_template
from flask_cors import CORS
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer

app = Flask(__name__)
CORS(app)

model_name = "facebook/blenderbot-400M-distill"
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)
conversation_history = []

@app.route('/chatbot', methods=['POST'])
def handle_prompt():
    # Read prompt from HTTP request body
    data = request.get_json()
    input_text = data["prompt"]

    # Keep only the recent conversation
    conversation_history[:] = conversation_history[-6:]

    # Create conversation history string
    history = "\n".join(conversation_history)
    prompt = history + f"\nUser: {input_text}\nBot:"

    # Tokenize the input
    inputs = tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True,
        max_length=512
    )

    # Generate the response
    outputs = model.generate(
        **inputs,
        max_new_tokens=60,
        no_repeat_ngram_size=3,
        repetition_penalty=1.3,
        do_sample=True,
        temperature=0.6,
        top_p=0.85
    )

    # Decode the response
    response = tokenizer.decode(outputs[0], skip_special_tokens=True).stri

    # Add interaction to conversation history
```

The content of this lab is licensed under Apache 2.0

```
    return response

if __name__ == '__main__':
    app.run()
```

Ahora probemos tu implementación usando `curl` para hacer una solicitud POST a /chatbot con el siguiente cuerpo de solicitud: {'prompt':'Hola, ¿cómo estás hoy?'}.

Abre una nueva terminal: Selecciona la pestaña de terminal --> abre una nueva terminal.

bash

```
curl -X POST -H "Content-Type: application/json" -d '{"prompt": "Hello, ho
```

Run

Aquí está la salida del código anterior:

txt

```
Hello, I am doing well. How are you? What do you do for a living?
```

¡Si recibiste una respuesta similar, entonces felicidades! ¡Has creado con éxito un servidor backend de Flask con un chatbot integrado!

Después de terminar, presiona `Ctrl + c` para detener el servidor.

The content of this lab is licensed under Apache 2.0

Comunicarse con tu backend usando una página web

En esta sección, descargarás una página web de chatbot en plantilla y la configurarás para hacer solicitudes a tu servidor backend.

Primero, clonemos un repositorio que tiene un sitio web de plantilla e instalemos las bibliotecas necesarias. Si tu aplicación Flask está en ejecución, termínala con `Ctrl + C` y ejecuta las siguientes líneas en la terminal:

bash

```
git clone https://github.com/ibm-developer-skills-network/LLM_application_
python3.11 -m pip install -r LLM_application_chatbot/requirements.txt
```

Run

Si las operaciones se completan sin errores, entonces has obtenido con éxito una copia del repositorio de plantillas.

La estructura de archivos de este repositorio debería ser la siguiente:

- LLM_application_chatbot/
  - static/
    - script.js
    - < otros activos >
  - templates/
    - index.html

Vamos a mover tu aplicación flask `app.py` a la carpeta `LLM_application_chatbot/` para que puedas alojar `index.html` en tu servidor.

Tanto `app.py` como la carpeta `LLM_application_chatbot/` deberían estar en `/home/project`. Puedes mover `app.py` a `LLM_application_chatbot/` ejecutando

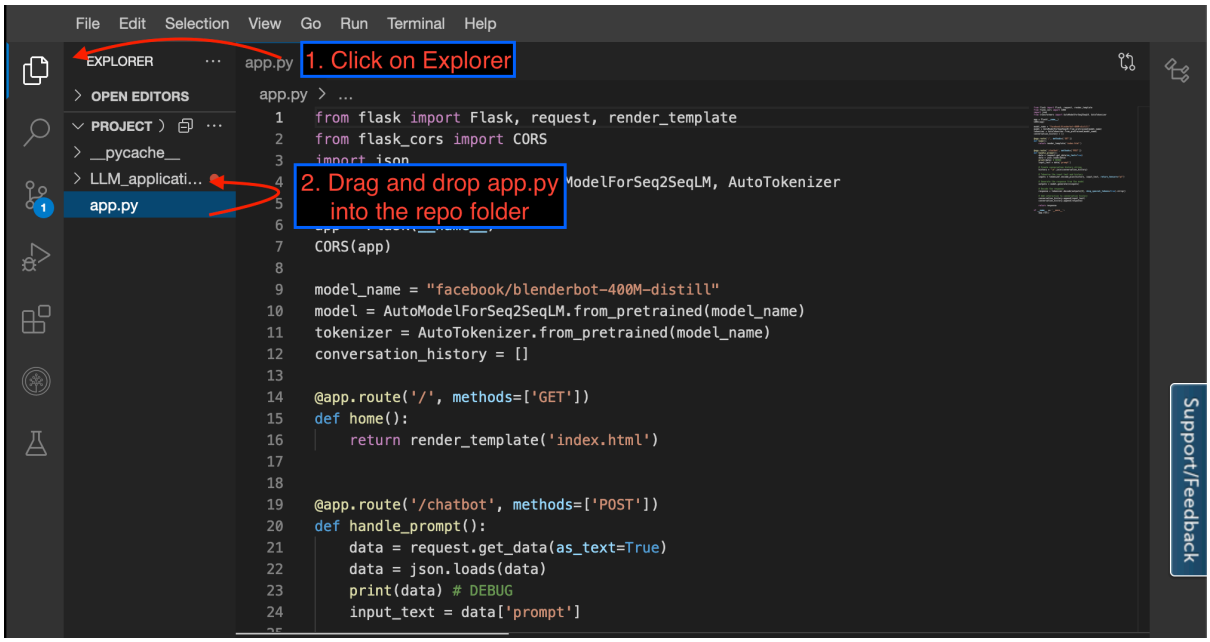
The content of this lab is licensed under Apache 2.0

```bash
mv app.py LLM_application_chatbot/
```

Run

Alternativamente, puedes hacer lo mismo arrastrando y soltando en el IDE de la siguiente manera:

1. Navega al Explorador desde la barra lateral izquierda
2. Arrastra y suelta `app.py` en `LLM_application_chatbot/` (Ten cuidado de no soltarlo en una subcarpeta)



Después de agregar tu aplicación flask, la estructura de archivos debería ser la siguiente:

- LLM_application_chatbot/
  - *app.py*
  - static/
    - script.js
    - < otros recursos >

The content of this lab is licensed under Apache 2.0

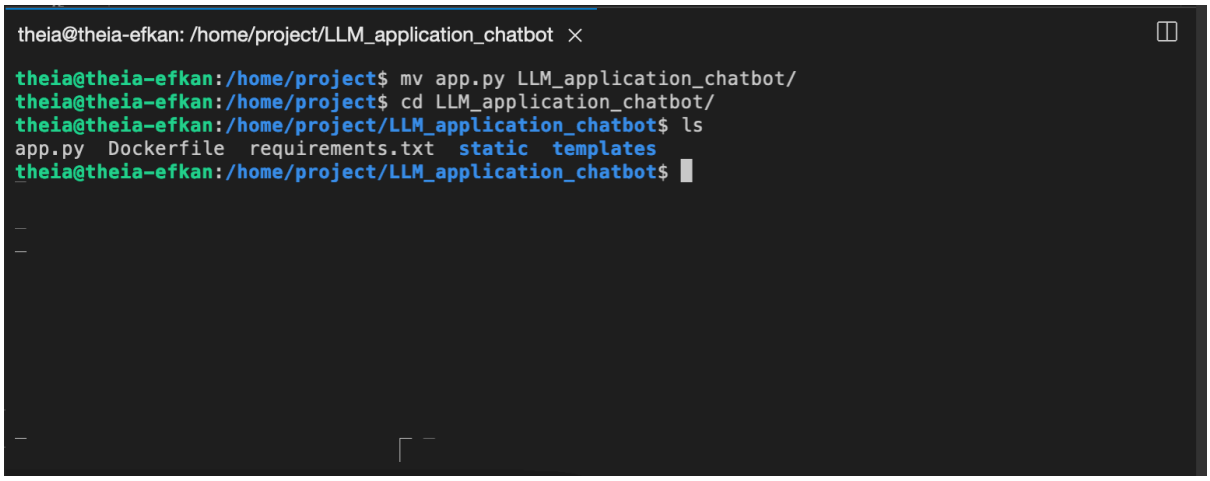
- index.html

Probemos esto ejecutando:

```bash
cd LLM_application_chatbot/
ls
```

Run

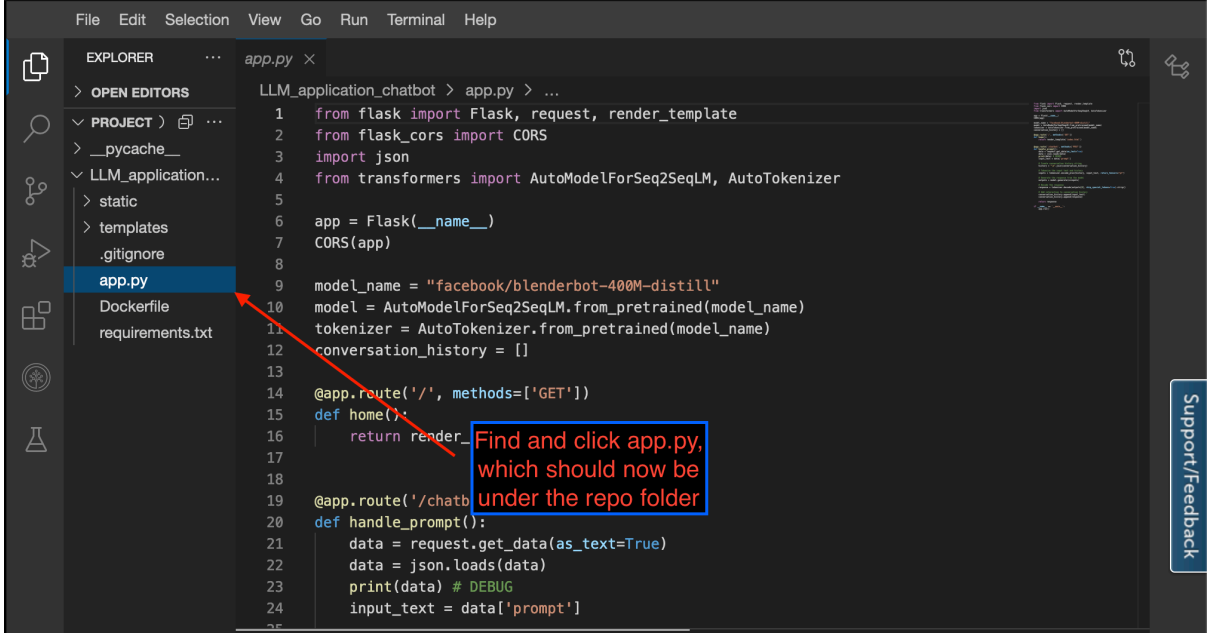
Deberías ver `app.py` en la salida:



Dado que has cambiado la ubicación de `app.py`, debes volver a abrirlo en tu editor. Esto se debe a que la pestaña actual para `app.py` intenta leer y escribir en app.py en una ruta que ya no existe.

Simplemente cierra la pestaña del editor para `app.py` y vuelve a abrirlo haciendo clic en `app.py` en el explorador.

The content of this lab is licensed under Apache 2.0



Ahora, modifiquemos tu `app.py` para que sirvas index.html en /. Puedes lograr esto añadiendo la siguiente ruta a tu código:

```py
@app.route('/', methods=['GET'])
def home():
    return render_template('index.html')
```

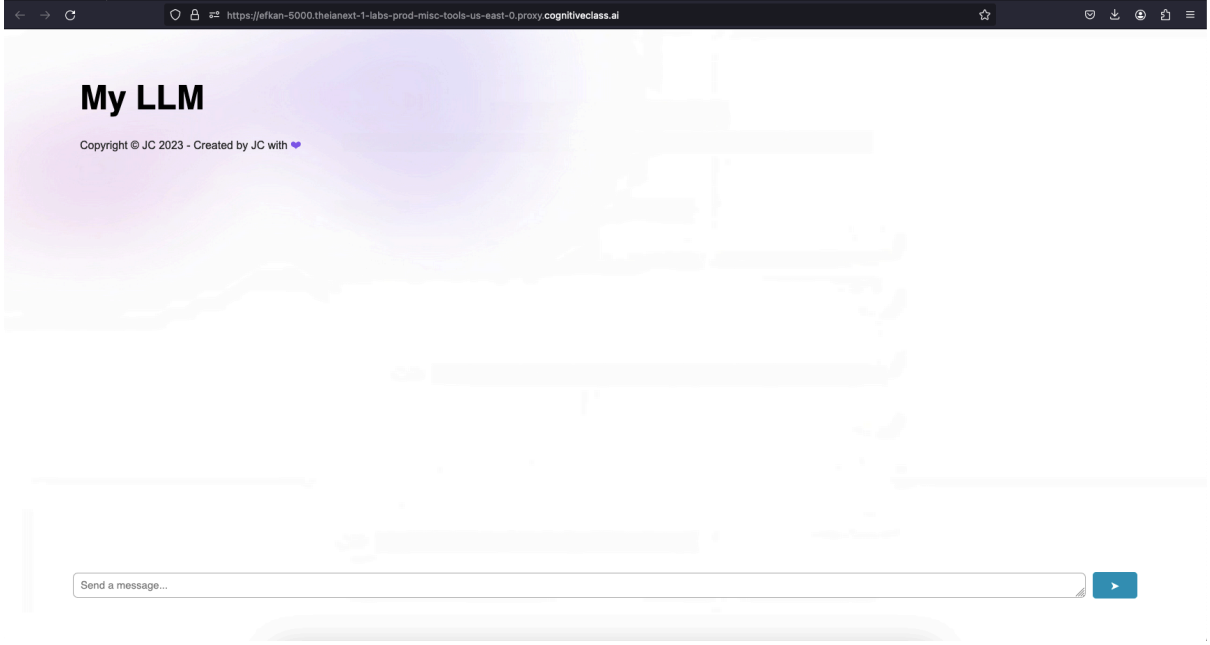
Después de agregar el código, puedes ejecutar tu aplicación flask con lo siguiente:

```bash
flask run
```

Una vez que tu servidor Flask esté en funcionamiento, puedes ver el renderizado de `index.html` visitando /.

Esto es lo que deberías ver:

The content of this lab is licensed under Apache 2.0



Esta plantilla ya tiene código JavaScript que emula una interfaz de chatbot. Cuando escribes un mensaje y presionas enviar, la plantilla del sitio web hace lo siguiente:

1. Introduce tu mensaje de entrada en un globo de texto
2. Envía tu entrada a un endpoint (por defecto, esto está configurado en www.example.com en la plantilla)
3. Espera la respuesta del endpoint y coloca la respuesta en un globo de texto

No implementarás tal interfaz ya que está fuera del propósito de este laboratorio.

En su lugar, te asegurarás de que en el paso 2, envíes la entrada del usuario a la ruta que creaste para tu chatbot anteriormente: http://127.0.0.1:5000/chatbot.

Para enviar la entrada, abres `/static/script.js` y encuentras dónde se establece el endpoint.

The content of this lab is licensed under Apache 2.0

```
const sendMessage = async (message) => {
  // addMessage(message, 'user','user.jpeg');
  addMessage(message, 'user','../static/user.jpeg');
  // Loading animation
  const loadingElement = document.createElement('div');
  const loadingtextElement = document.createElement('p');
  loadingElement.className = `loading-animation`;
  loadingtextElement.className = `loading-text`;
  loadingtextElement.innerText = 'Loading....Please wait';
  messagesContainer.appendChild(loadingElement);
  messagesContainer.appendChild(loadingtextElement);

  async function makePostRequest(msg) {
    const url = 'www.example.com';
    const requestBody = {
      prompt: msg
    };

    try {
      const response = await fetch(url, {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json'
        },
        body: JSON.stringify(requestBody)
      });

      const data = await response.text();
      // Handle the response data here
      console.log(data);
      return data;
    } catch (error) {
      // Handle any errors that occurred during the request
      console.error('Error:', error);
      return error
    }
  }
```

Cambiemos este punto final a la ruta de tu chatbot. En la plantilla, reemplaza el punto final predeterminado (www.example.com) con la URL de tu backend de chatbot.

plaintext

```
'https://sinanz-5000.theianext-0-labs-prod-misc-tools-us-east-0.proxy.cogn
```

The content of this lab is licensed under Apache 2.0

La URL puede ser diferente para ti. Básicamente, copia la URL en el lanzamiento de tu aplicación y añade `/chatbot` al final.



¡Y eso debería ser todo! Antes de probar tu código, echemos un vistazo a la versión final de tu aplicación Flask:

The content of this lab is licensed under Apache 2.0

py

```python
from flask import Flask, request, render_template
from flask_cors import CORS
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer

app = Flask(__name__)
CORS(app)

model_name = "facebook/blenderbot-400M-distill"
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)
conversation_history = []

@app.route('/', methods=['GET'])
def home():
    return render_template('index.html')

@app.route('/chatbot', methods=['POST'])
def handle_prompt():
    data = request.get_json()
    print(data)  # DEBUG
    input_text = data["prompt"]

    # Keep only the recent conversation
    conversation_history[:] = conversation_history[-6:]

    # Create conversation history string
    history = "\n".join(conversation_history)
    prompt = history + f"\nUser: {input_text}\nBot:"

    # Tokenize the input
    inputs = tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True,
        max_length=512
    )

    # Generate the response
    outputs = model.generate(
        **inputs,
        max_new_tokens=60,
        no_repeat_ngram_size=3,
        repetition_penalty=1.3,
        do_sample=True,
        temperature=0.6,
        top_p=0.85
    )
```

The content of this lab is licensed under Apache 2.0

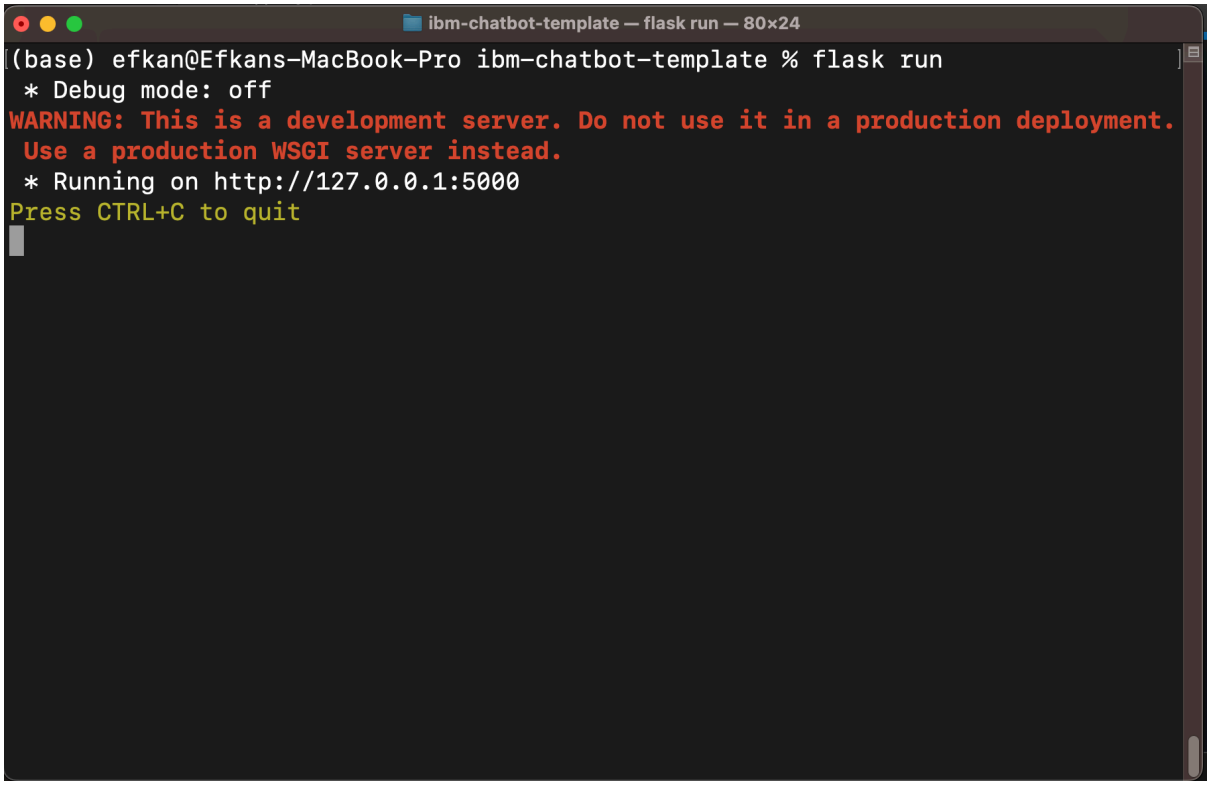
```
    # Add interaction to conversation history
    conversation_history.append(f"User: {input_text}")
    conversation_history.append(f"Bot: {response}")

    return response

if __name__ == '__main__':
    app.run()
```

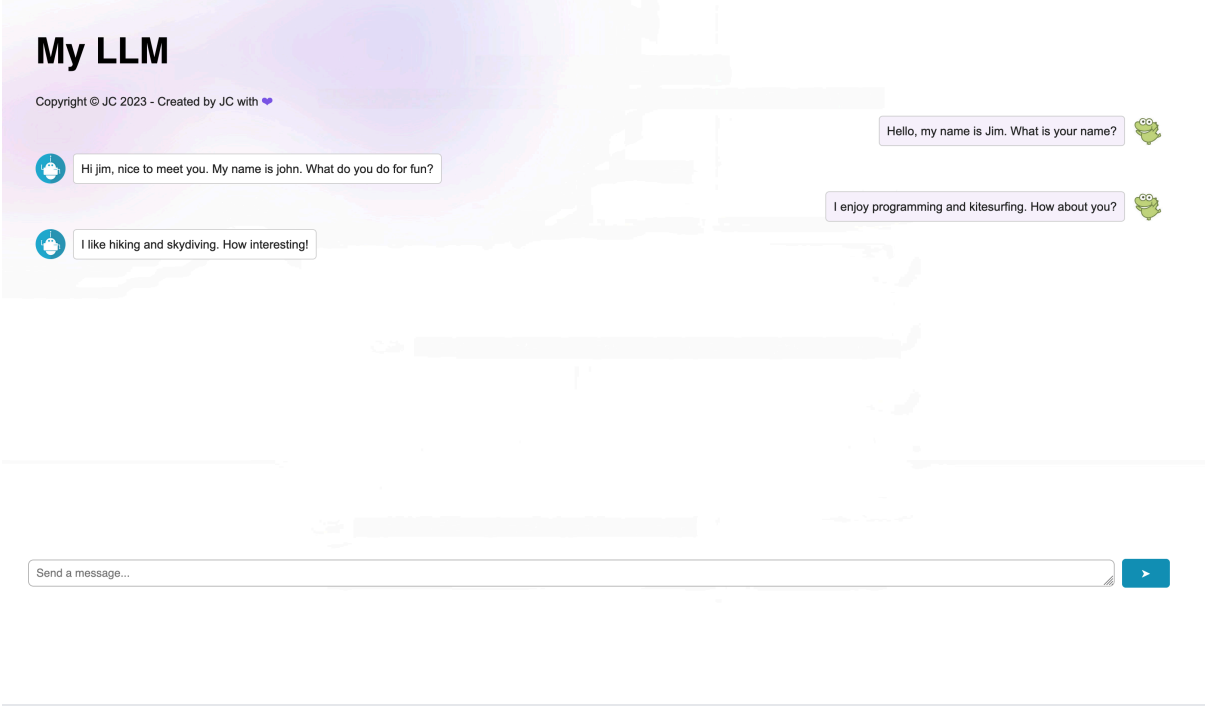
¡Está bien! Ahora probemos tu aplicación deteniéndola primero con `Ctrl + C` (si está en ejecución) y reiniciándola con `flask run`.

En tu terminal, deberías ver algo como esto:



¡Y ahora naveguemos a tu página de inicio en / y probemos tu chatbot!

The content of this lab is licensed under Apache 2.0



¡Felicidades! Si tu resultado es similar, ¡entonces acabas de crear tu propio sitio web de chatbot!

Author

Sina Nazeri (Ph.D.) linkedin

© IBM Corporation. Todos los derechos reservados.

Integrando tu Chatbot en una Interfaz Web

Skills Network

The content of this lab is licensed under Apache 2.0

Introducción

En este laboratorio, aprenderás a configurar un servidor de backend e integrar tu chatbot en una aplicación web.

Objetivos de aprendizaje

Después de completar este laboratorio, podrás:

- Configurar tu servidor de backend
- Integrar tu chatbot en tu servidor Flask
- Comunicarte con el backend utilizando una página web

Requisitos previos

Esta sección asume que sabes cómo construir el chatbot simple de terminal explicado en el primer laboratorio.

Hay dos cosas que debes construir para crear tu sitio web similar a ChatGPT:

1. Un servidor de backend que aloje tu chatbot
2. Una página web de frontend que se comunique con tu servidor de backend

Sin más preámbulos, ¡comencemos!

Paso 1: Alojar tu chatbot en un servidor de backend

¿Qué es un servidor de backend?

Un servidor de backend es como el cerebro detrás de un sitio web o aplicación. En este caso, el servidor de backend recibirá solicitudes de tu sitio web, las alimentará a tu chatbot y devolverá la salida del chatbot de nuevo al sitio web, que será leído por el usuario.

Alojar un servidor de backend simple usando Flask

The content of this lab is licensed under Apache 2.0

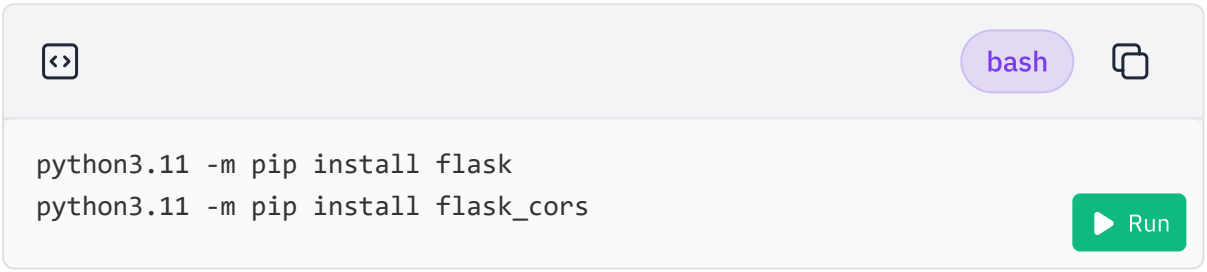
`flask` es un marco de trabajo de Python para construir aplicaciones web con Python. Proporciona un conjunto de herramientas y funcionalidades para manejar solicitudes entrantes, procesar datos y generar respuestas, facilitando el funcionamiento de tu sitio web o aplicación.

Requisitos previos

Para todas las interacciones en la terminal en este laboratorio (como ejecutar archivos de python o instalar paquetes), utilizarás la terminal integrada que viene con Cloud IDE. Puedes abrir la terminal presionando:

- `Ctrl + ``, o
- Seleccionando Terminal --> Nueva Terminal en la barra de herramientas en la parte superior de la ventana del IDE a la derecha.

En tu terminal, instalemos los siguientes requisitos:



Configurando el servidor

A continuación, crearás un script que almacene tu código del servidor flask.

Para crear un nuevo archivo de Python, haz clic en `File Explorer`, luego haz clic derecho en el área del explorador y selecciona `New File`. Nombra este nuevo archivo `app.py`.

Veamos cómo implementar un servidor flask simple:

The content of this lab is licensed under Apache 2.0

```py
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return 'Hello, World!'

if __name__ == '__main__':
    app.run()
```

Pega el código anterior en el archivo `app.py` que acabas de crear y guárdalo.

En este código:

- Importas la clase `Flask` del módulo `flask`.
- Creas una instancia de la clase `Flask` y la asignas a la variable `app`.
- Defines una ruta para la página de inicio decorando la función `home()` con el decorador `@app.route()`. La función devuelve la cadena '¡Hola, Mundo!'. Esto significa que cuando el usuario visita la URL donde está alojado el sitio web, el servidor backend recibirá la solicitud y devolverá '¡Hola, Mundo!' al usuario.
- La condición `if __name__ == '__main__':` asegura que el servidor solo se ejecute si el script se ejecuta directamente, no cuando se importa como un módulo.
- Finalmente, llamas a `app.run()` para iniciar el servidor.

```bash
python3.11 app.py
```

Run

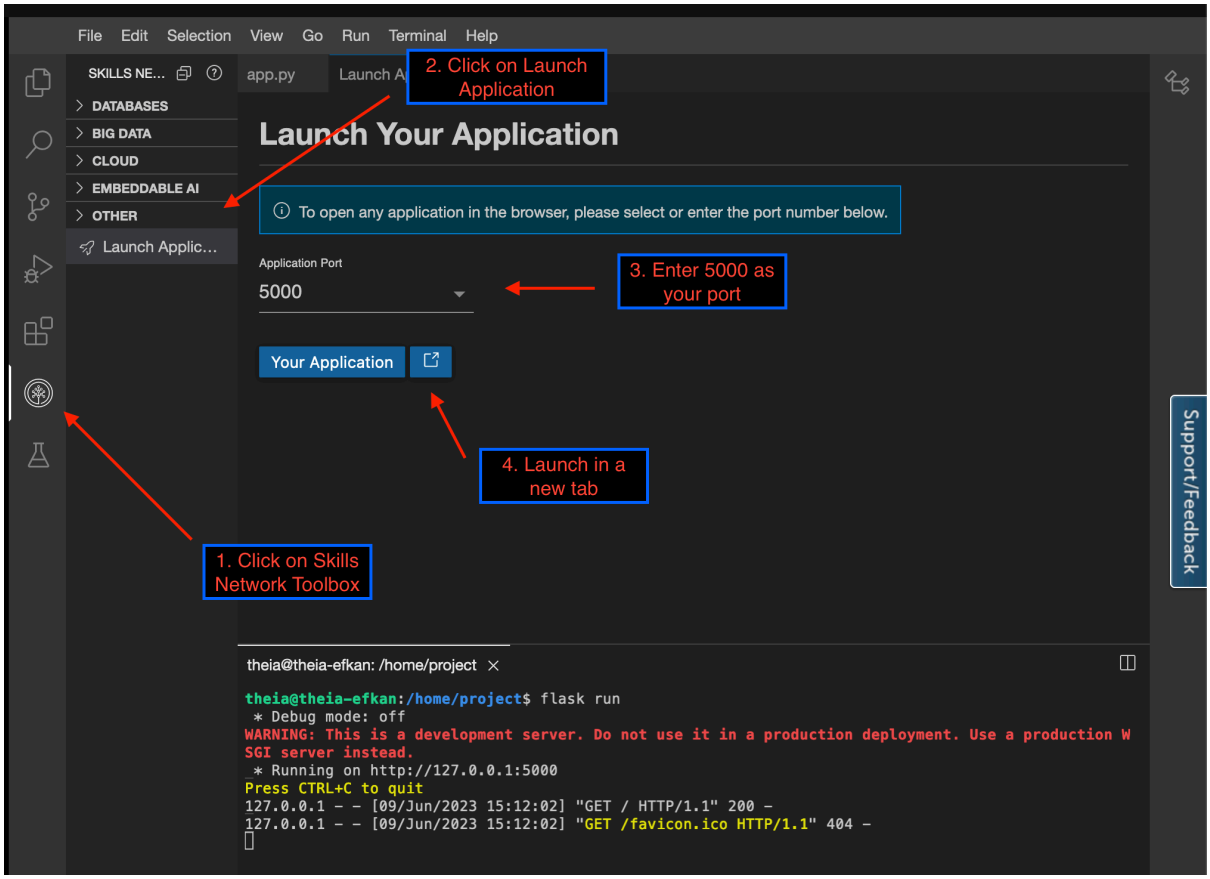
Guarda este código en un archivo de Python, por ejemplo, app.py, y ejecútalo escribiendo `python app.py` en la terminal. Por defecto, Flask aloja el servidor en http://127.0.0.1:5000/ (que es equivalente a http://127.0.0.1:5000/).

Con este comando, el servidor Flask comenzará a ejecutarse. Si ejecutas este

The content of this lab is licensed under Apache 2.0

Sin embargo, actualmente estás ejecutando este laboratorio en la Nube de Skills Network. Por lo tanto, puedes acceder a tu servidor de la siguiente manera:

1. Navega a la Caja de Herramientas de Skills Network desde la barra de herramientas en el lado izquierdo del IDE
2. Haz clic en "Lanzar Aplicación" en la barra lateral vertical adyacente
3. Ingresa 5000 como tu Puerto de Aplicación
4. Lanza la aplicación en una nueva pestaña del navegador



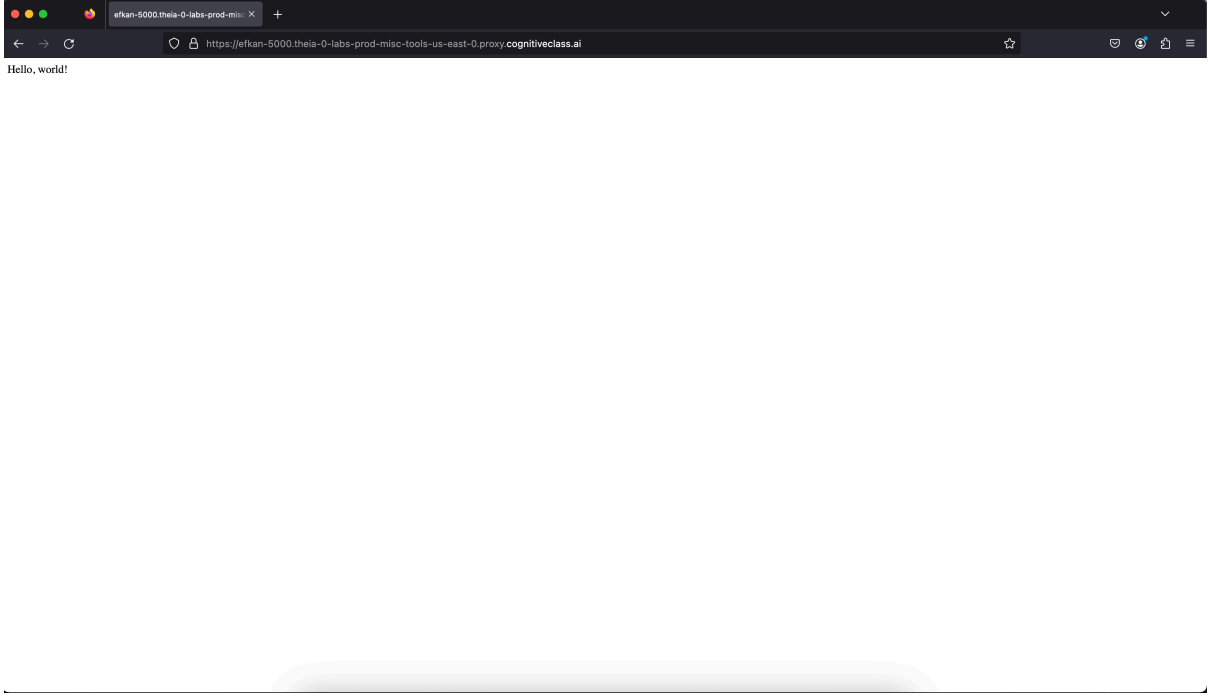
Al realizar los pasos anteriores, has visitado la **URL relativa de localhost** del servidor en la nube.

IMPORTANTE: A lo largo del resto de este laboratorio, te referirás a esta URL como .

Al visitar el localhost, deberías ver el mensaje "¡Hola, Mundo!" mostrado.

Así es como debería verse:

The content of this lab is licensed under Apache 2.0



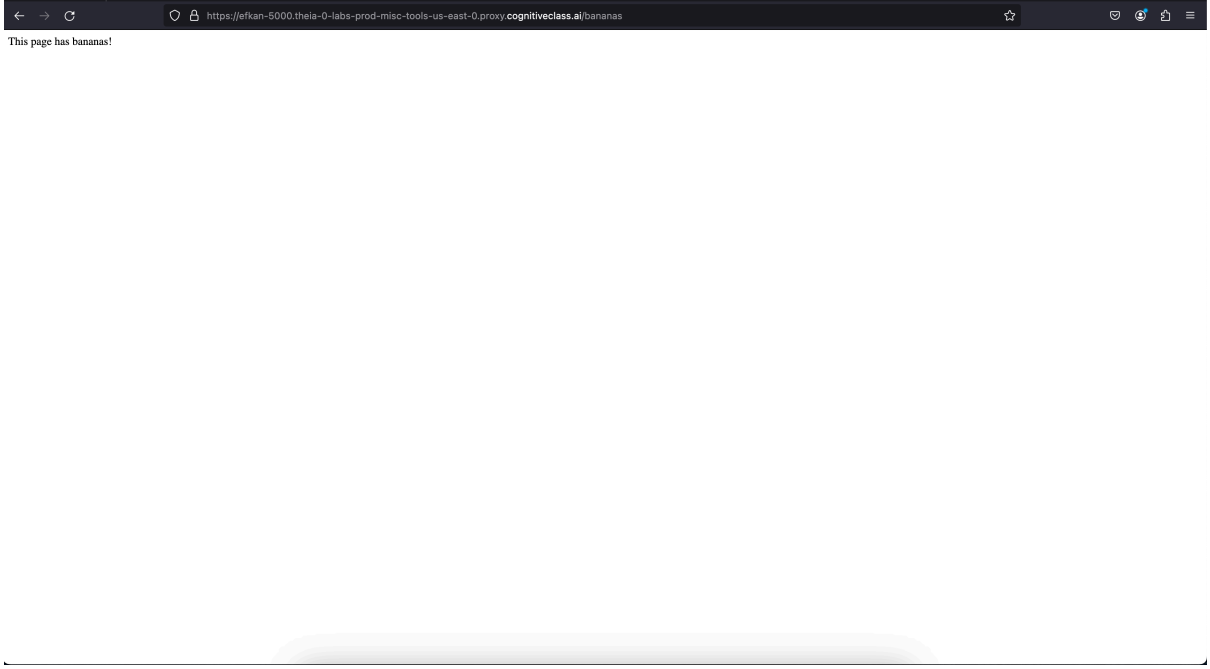
Agreguemos las siguientes rutas para probarlo:

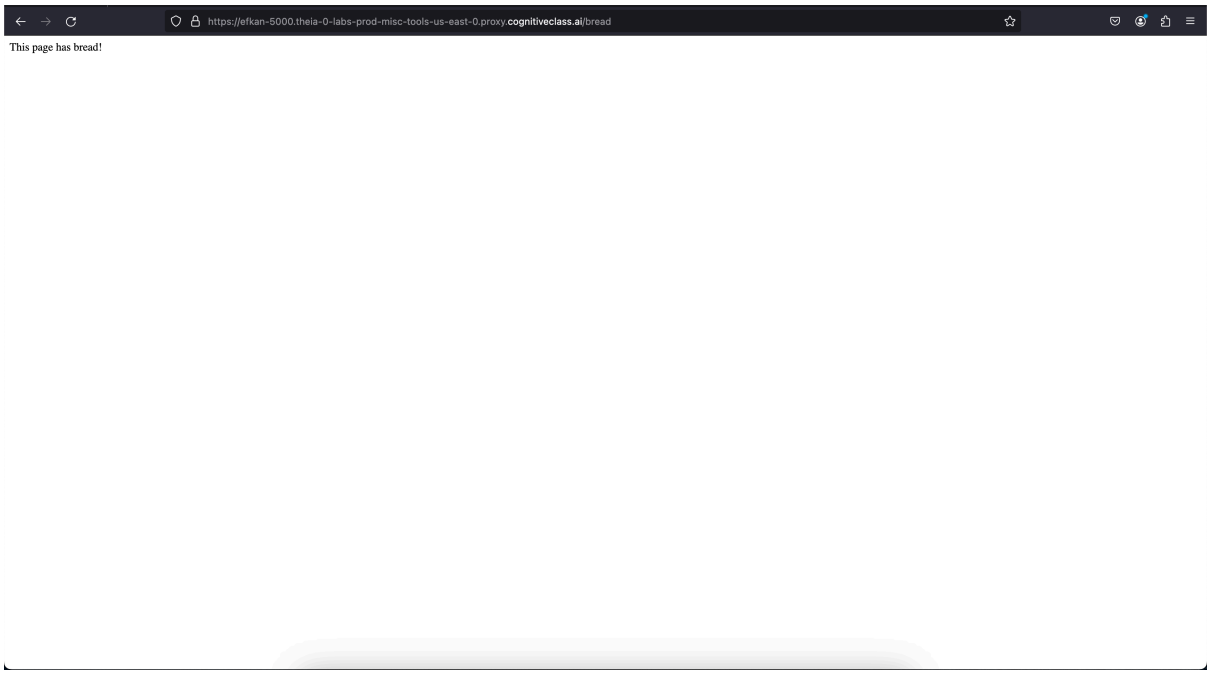
```py
@app.route('/bananas')
def bananas():
    return 'This page has bananas!'

@app.route('/bread')
def bread():
    return 'This page has bread!'
```

Ahora, detengamos nuestra aplicación usando `Ctrl + C` en la terminal y volvamos a ejecutar con `flask run`. Luego, visitemos ambas rutas en /bananas y /bread. Esto es lo que deberías ver:

The content of this lab is licensed under Apache 2.0





Está bien, ahora que has demostrado cómo funcionan las rutas, puedes eliminar estas dos rutas (bananas y bread) de tu `app.py` ya que no las estarás utilizando.

Antes de continuar, también agregarás dos líneas más de código a tu programa para mitigar los errores de CORS, un tipo de error relacionado con hacer solicitudes a dominios diferentes al que aloja esta página web. Modificarás tu código de la siguiente manera:

The content of this lab is licensed under Apache 2.0

```py
from flask import Flask
from flask_cors import CORS     # newly added

app = Flask(__name__)
CORS(app)               # newly added

@app.route('/')
def home():
    return 'Hello, World!'

if __name__ == '__main__':
    app.run()
```

Integrando tu chatbot en tu servidor Flask

Ahora que tienes tu servidor Flask configurado, integremos tu chatbot en tu servidor Flask.

Como se indicó al principio, este laboratorio asume que has completado el primer laboratorio de este proyecto guiado sobre cómo crear tu propio chatbot simple.

Primero, instalarás los requisitos. Fijamos las versiones de las bibliotecas para asegurar que el código se ejecute de manera consistente sin romperse debido a actualizaciones futuras o cambios en las dependencias.

```plaintext
python3.11 -m pip install transformers==4.41.2
python3.11 -m pip install torch==2.2.2
python3.11 -m pip install accelerate==0.30.1
python3.11 -m pip install numpy==1.26.4
```

A continuación, copia el código para inicializar tu chatbot del laboratorio 1 y colócalo en la parte superior de tu script. También debes importar las bibliotecas necesarias para tu chatbot.

```py
from transformers import AutoModelForSeq2SeqLM
from transformers import AutoTokenizer
```

The content of this lab is licensed under Apache 2.0

```py
model_name = "facebook/blenderbot-400M-distill"
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)
conversation_history = []
```

A continuación, necesitarás importar un módulo para leer los datos.

```py
from flask import request
```

Antes de implementar la función real, necesitas determinar la estructura que esperas recibir en la solicitud HTTP entrante.

Definamos tu estructura esperada de la siguiente manera:

```json
{
    "prompt": "message"
}
```

Ahora implementa la función de tu chatbot. Nuevamente, copiarás el código de tu implementación de chatbot del primer laboratorio.

The content of this lab is licensed under Apache 2.0

```py
@app.route('/chatbot', methods=['POST'])
def handle_prompt():
    data = request.get_json()
    input_text = data["prompt"]

    # Keep only recent conversation
    conversation_history[:] = conversation_history[-6:]

    # Create conversation history string
    history = "\n".join(conversation_history)
    prompt = history + f"\nUser: {input_text}\nBot:"

    # Tokenize input
    inputs = tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True,
        max_length=512
    )

    # Generate response
    outputs = model.generate(
        **inputs,
        max_new_tokens=60,
        no_repeat_ngram_size=3,
        repetition_penalty=1.3,
        do_sample=True,
        temperature=0.6,
        top_p=0.85
    )

    # Decode response
    response = tokenizer.decode(outputs[0], skip_special_tokens=True).stri

    # Save conversation
    conversation_history.append(f"User: {input_text}")
    conversation_history.append(f"Bot: {response}")

    return response
```

La única novedad que has hecho hasta ahora es leer el mensaje de la solicitud HTTP. ¡Has copiado todo lo demás de tu implementación anterior del chatbot!

Perfecto, ahora antes de probar tu aplicación, así es como se ve la versión final de

The content of this lab is licensed under Apache 2.0

py

```
from flask import Flask, request, render_template
from flask_cors import CORS
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer

app = Flask(__name__)
CORS(app)

model_name = "facebook/blenderbot-400M-distill"
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)
conversation_history = []

@app.route('/chatbot', methods=['POST'])
def handle_prompt():
    # Read prompt from HTTP request body
    data = request.get_json()
    input_text = data["prompt"]

    # Keep only the recent conversation
    conversation_history[:] = conversation_history[-6:]

    # Create conversation history string
    history = "\n".join(conversation_history)
    prompt = history + f"\nUser: {input_text}\nBot:"

    # Tokenize the input
    inputs = tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True,
        max_length=512
    )

    # Generate the response
    outputs = model.generate(
        **inputs,
        max_new_tokens=60,
        no_repeat_ngram_size=3,
        repetition_penalty=1.3,
        do_sample=True,
        temperature=0.6,
        top_p=0.85
    )

    # Decode the response
    response = tokenizer.decode(outputs[0], skip_special_tokens=True).stri

    # Add interaction to conversation history
```

The content of this lab is licensed under Apache 2.0

```
    return response

if __name__ == '__main__':
    app.run()
```

Ahora probemos tu implementación usando `curl` para hacer una solicitud POST a /chatbot con el siguiente cuerpo de solicitud: {'prompt':'Hola, ¿cómo estás hoy?'}.

Abre una nueva terminal: Selecciona la pestaña de terminal --> abre una nueva terminal.

bash

```
curl -X POST -H "Content-Type: application/json" -d '{"prompt": "Hello, ho
```

Run

Aquí está la salida del código anterior:

txt

```
Hello, I am doing well. How are you? What do you do for a living?
```

¡Si recibiste una respuesta similar, entonces felicidades! ¡Has creado con éxito un servidor backend de Flask con un chatbot integrado!

Después de terminar, presiona `Ctrl + c` para detener el servidor.

Comunicarse con tu back… →

The content of this lab is licensed under Apache 2.0